OptSQL: A Meta-Cognitive Multi-Agent System for Efficiency-Oriented Text-to-SQL

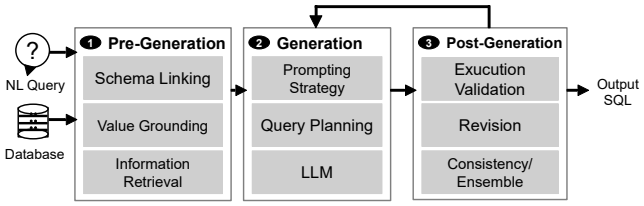

Fig. 1: **Overview of a typical Text-to-SQL pipeline, illustrating the Pre-Generation, Generation, and Post-Generation stages**

***Abstract*—Pending...**

I. Introduction

Text-to-SQL aims to translate natural language (NL) questions into executable SQL queries, enabling non-expert users to retrieve and analyze relational data without manually writing database programs [1], [2], [3]. Recent LLM-based systems have achieved substantial progress through in-context learning, task decomposition and specialized agents, moving the field beyond single-shot generation toward workflow-based SQL construction [4], [5], [6], [7].

From Single-shot Generation to Agentic Workflows. As shown in Figure 1, modern Text-to-SQL systems typically organize SQL construction into three stages: pre-generation, generation, and post-generation. In the pre-generation stage, the system grounds the natural language query in the database context through schema linking, value grounding, and information retrieval, thereby reducing the search space and enriching the input evidence [7], [8], [9], [10]. In the generation stage, the model applies prompting strategies, query planning, and LLM-based reasoning to produce one or more candidate SQL queries [4], [5], [6], [11], [12], [13]. In the post-generation stage, these candidates are further checked and refined through execution validation, repair, and consistency- or ensemble-based selection [14], [5], [15], [6], [7], [11], [12], [13]. Compared with single-pass generation, such workflows are more robust because they decompose SQL construction into modular steps, strengthen database grounding, diversify candidate generation, and provide post-hoc opportunities for validation and refinement. Despite these advances, existing Text-to-SQL systems still face three coupled limitations:

L1: Limited Adaptation in Workflows and Strategy Selection. Many existing Text-to-SQL agents often predefine both the workflow structure and the strategy choices, where modules such as schema linking, few-shot retrieval, candidate generation, validation, and revision are applied in a largely fixed manner [4], [5], [6], [11], [9]. Although such designs improve robustness over single-shot generation, they offer limited support for adapting reasoning strategies to the database environment, question difficulty, or intermediate feedback. This may misallocate reasoning effort: simple queries over intuitive schemas can incur unnecessary token and latency overhead, whereas difficult queries may require deeper grounding, candidate comparison, and additional validation or reflection. This mismatch is illustrated by the different effects of domain-specific hints in BIRD [16] dataset. For an intuitive database such as `superhero`, where tables and columns are relatively descriptive, adding domain hints provides little additional benefit in our experiments. In contrast, for a domain-heavy database such as `thrombosis_prediction`, where terms such as `LDH` and gender-dependent uric-acid ranges require medical knowledge, the same hints improve accuracy from 20% to nearly 50%. This contrast suggests that Text-to-SQL agents should adapt their grounding and generation strategies to the database environment, rather than applying the same strategy uniformly.

L2: Post-hoc rather than Process-level Reflection. Feedback in many Text-to-SQL systems is mainly used after a candidate SQL has been generated, such as repairing execution errors, revising invalid queries, or selecting among completed candidates [5], [6], [7], [17]. Such feedback improves final-output validation, but provides limited support for monitoring and regulating the generation process itself. In particular, the agent often lacks a controller that continuously tracks whether schema linking is reliable, whether candidate disagreement is increasing, whether repeated repairs are converging, or whether the current strategy should be changed. As a result, early mistakes in schema linking, value grounding, or query planning may propagate into later steps and become cascading hallucinations. Moreover, without global process supervision, local repair actions may fix surface-level execution errors while leaving the overall reasoning trajectory insufficiently grounded in the user question and database context.

L3: Limited Awareness of Query Efficiency. Most existing Text-to-SQL systems are primarily optimized for correctness: they aim to generate SQL queries whose execution results match the ground-truth answers [18], [19], [20], [21], [22], [23]. However, in practical database applications, a semantically correct SQL query is not necessarily efficient. Equivalent SQL formulations may lead to very different execution costs due to variations in join orders, predicate placement, or aggregation strategies [24], [25], [26]. Yet, current Text-to-SQL systems mainly focus on executability and result consistency,

rarely treating query plans, estimated costs, or bottleneck patterns as signals for rewriting generated SQL. As a result, the final query may be correct but unnecessarily expensive, especially in enterprise settings with large schemas, large tables, and long multi-step analytical workflows [27], [17].

Key Insights: Meta-cognitive and Environment-aware Text-to-SQL. Inspired by enactive cognition and meta-cognition, we view Text-to-SQL not as a static translation from question and schema to SQL, but as an adaptive interaction process between the agent and the database environment [28], [29]. Enactive cognition suggests that understanding is shaped through interaction with the environment, while meta-cognition emphasizes the ability to monitor and regulate one's own reasoning process. In the context of Text-to-SQL, this means that an agent should perceive database-specific signals, act by generating and rewriting SQL, observe feedback from execution and query plans, and adjust its strategy when uncertainty, failure, or inefficiency is detected. These perspectives lead to three design objectives. First, to address **L1**, the agent should perform *environment-aware dynamic workflow adaptation*: it should select lightweight or deep workflows according to the perceived database environment, question complexity, and intermediate feedback, rather than applying the same strategy uniformly. Second, to address **L2**, the agent should support *process-level self-reflection and dynamic control*: it should monitor schema-linking reliability, candidate disagreement, repair convergence, and strategy suitability during generation, so that early mistakes can be corrected before they propagate. Third, to address **L3**, the agent should enable *execution- and plan-aware SQL optimization*: it should use query plans, cost signals, semantic validation, and reusable rewrite knowledge to improve query efficiency while preserving correctness.

Approach. To realize these objectives, we propose OPTSQL, a meta-cognitive multi-agent system for efficiency-oriented Text-to-SQL. At the core of OPTSQL is a Meta-Cognitive Controller that maintains a task-level state in working memory, observes intermediate feedback, and dynamically decides which agent actions should be invoked next. Under its control, the Generation Phase constructs executable and semantically plausible initial SQL candidates through evidence-guided schema filtering and multi-strategy SQL generation. When further efficiency improvement is needed, the Optimization Loop analyzes query plans, identifies bottleneck patterns, rewrites inefficient SQL formulations, and validates both semantic consistency and cost reduction. Finally, a Memory Engine and Tool Infrastructure support database grounding, execution analysis, retrieval of reusable optimization experience, and updates to the long-term Auto-Evaluation Knowledge Base (AEKB) after successful rewrites.

II. MOTIVATION

A. Task Formulation

Traditional Text-to-SQL aims to translate a NL question x into an executable SQL query q over a database environment $\mathcal{D}$. Given a ground-truth query q^*, the standard objective is to generate a query whose execution result matches that of q^*:

$$f_\theta(x, \mathcal{D}) \to q, \quad \text{s.t.} \quad \text{Exec}(q, \mathcal{D}) = \text{Exec}(q^*, \mathcal{D}),$$

where f_θ denotes a Text-to-SQL system parameterized by θ, $\mathcal{D}$ denotes the database environment including schema, data, indexes, and statistics, and $\text{Exec}(q, \mathcal{D})$ denotes the result of executing q on $\mathcal{D}$.

However, this formulation focuses mainly on semantic correctness and does not explicitly consider query efficiency. In practice, multiple semantically equivalent SQL queries may produce the same result but incur substantially different execution costs due to join order, predicate placement, aggregation strategy, or index utilization. We therefore formulate **Efficiency-Oriented Text-to-SQL**, where the goal is to generate a query that is both semantically correct and efficient. Given a NL question x and a database environment $\mathcal{D}$, the goal is to generate a SQL query q_{opt} satisfying:

$$q_{\text{opt}} = \arg \min_{q \in \mathcal{Q}_{\text{corr}}(x, \mathcal{D})} \text{Cost}(q, \mathcal{D}),$$
$$\mathcal{Q}_{\text{corr}}(x, \mathcal{D}) = \{q \in \mathcal{Q}(x, \mathcal{D}) \mid \text{Exec}(q, \mathcal{D}) = \text{Exec}(q^*, \mathcal{D})\},$$

where $\mathcal{Q}_{\text{corr}}(x, \mathcal{D})$ denotes the subset of candidates that are semantically correct with respect to q^*. $\text{Cost}(q, \mathcal{S})$ represents the estimated or actual execution cost of query q on $\mathcal{S}$. Intuitively, the model should not only ensure that q_{opt} yields correct results but also that it achieves minimal execution time or resource consumption under the same semantic constraints.

B. Design Objective

Existing Text-to-SQL systems are typically static pipelines that decompose SQL construction into pre-defined stages, including pre-generation, generation, and post-generation validation. While such designs improve robustness compared to single-pass generation, they lack the ability to dynamically adapt their reasoning process to different database environments, query difficulties, and intermediate execution feedback.

Beyond Static Pipelines: A Meta-Cognitive Multi-Agent Perspective. Inspired by enactive cognition and meta-cognitive theory, we reformulate Text-to-SQL as a dynamic interaction process between an intelligent system and the database environment [28], [29]. Enactive cognition emphasizes that intelligence emerges through continuous interaction with the environment, while meta-cognition enables the system to monitor, regulate, and adapt its own reasoning process. Based on these principles, we propose a Meta-Cognitive Multi-Agent Text-to-SQL System, where a central Meta-Cognitive Controller dynamically orchestrates a set of specialized agents (actions) under different execution contexts. Instead of a fixed pipeline, all stages in Text-to-SQL are decomposed into reusable and composable actions, which are invoked online based on the current state of the problem and environment feedback. This design establishes a unified closed-loop system in which a Meta-Cognitive Controller coordinates planning, execution, evaluation, and optimization over SQL programs based on continuous database feedback.

Key Design Principles. (1) Environment-aware Dynamic Workflow Adaptation. The Meta-Cognitive Controller continuously observes the evolving task state, such as database environment, question complexity, candidate behaviors and dynamically decides which actions should be invoked next. For example, for simple queries over well-structured schemas (e.g., single-table filtering), the controller may directly invoke a lightweight generation path using few-shot prompting. In contrast, for complex analytical queries (e.g., multi-join or domain-specific reasoning), the controller activates a deeper workflow that includes multi-candidate generation, iterative refinement, and optimization loops. This adaptive strategy selection avoids over- or under-utilization of reasoning resources.

(2) Process-level Self-Reflection and Dynamic Control. Unlike conventional post-hoc validation approaches, our system enables continuous monitoring of intermediate reasoning states during SQL generation. The Meta-Cognitive Controller uses the recorded task trace to diagnose whether the current reasoning trajectory is reliable, converging, and sufficiently grounded. For example, when schema or value evidence is insufficient, the controller may request additional evidence construction through reverse schema linking, value-grounded schema linking, or relational completion. Similarly, if repeated optimization iterations fail to reduce execution cost, the controller may terminate the current loop and switch to a different reasoning configuration. This enables closed-loop control over the entire reasoning trajectory.

(3) Execution- and Plan-aware Optimization via Feedback-driven Rewriting. Beyond generating semantically correct SQL, the system treats execution efficiency as a objective. Execution results, query plans, and cost estimates are used as feedback signals to reveal whether an initially correct query may still be inefficient. When such feedback indicates potential performance bottlenecks, the system can refine the SQL through feedback-driven rewriting while preserving its intended semantics. More importantly, optimization is not treated as an isolated one-shot action. The system accumulates validated optimization experience from previous interactions with the database environment, enabling future decisions to be guided not only by static rules but also by reusable execution-aware experience. In this way, SQL optimization becomes a closed-loop process in which database feedback continuously informs rewriting, validation, and future strategy selection.

III. OPTSQL

A. Overview

Figure 2 shows the overall architecture of OPTSQL. Given a NL question and a database environment, OPTSQL aims to generate a SQL query that is both semantically correct and execution-efficient. Instead of treating Text-to-SQL as a fixed pipeline, OPTSQL formulates it as a feedback-driven process coordinated by a Meta-Cognitive Controller. The Meta-Cognitive Controller supervises the entire workflow. It perceives the database environment, estimates the difficulty of the input question, selects suitable actions, and monitors intermediate feedback throughout both generation and optimization. Under its control, OPTSQL first constructs an initial SQL query in the Generation Phase, where an evidence-guided schema filter identifies relevant tables, columns, values, and join paths, and an initial SQL builder produces executable candidate queries. If the controller detects that the query requires further efficiency improvement, it activates the Optimization Loop. This loop consists of an Explain Analyzer Agent, a SQL Rewriter Agent, and a Validator Agent, which together analyze query plans, rewrite inefficient SQL patterns, and verify semantic consistency and cost reduction. OPTSQL is further supported by a Memory Engine and a Tool Infrastructure. The Memory Engine maintains task-level working memory and reusable optimization knowledge, including expert rules and validated historical rewrite cases. The Tool Infrastructure provides database grounding, SQL execution and analysis and RAG-based retrieval. Through these components, OPTSQL continuously observes feedback from the database environment, reflects on its intermediate decisions, and updates its optimization experience.

The remainder of this section describes each component of OPTSQL. We first present the Generation Phase for constructing an initial SQL query (Section III-B), followed by the Optimization Loop for execution- and plan-aware rewriting (Section III-C). We then introduce the Meta-Cognitive Controller that orchestrates these processes through strategy selection, monitoring, and self-reflection (Section III-D). Finally, we describe the Memory Engine and Tool Infrastructure that support retrieval, experience accumulation, and database feedback (Section III-E).

B. Generation Phase

The Generation Phase aims to construct a set of executable and semantically plausible initial SQL candidates based on the question and database environment. It focuses on two core capabilities: evidence construction and multi-strategy SQL generation.

Evidence-guided Schema Filter Agent. This agent is responsible for grounding the input question into a compact, relevant, and executable schema context for downstream SQL generation. It combines robust schema linking with value grounding, aiming to identify not only the tables and columns mentioned or implied by the question, but also the concrete database values and conditions needed to express the user intent. To improve schema robustness, the agent supports multiple complementary schema linking strategies:

- *Direct schema linking* maps the natural language question to relevant tables and columns by reasoning over question semantics and schema descriptions. This strategy is effective when the question contains explicit lexical or semantic cues.
- *Reverse schema linking* hypothesizes a tentative SQL structure and then extracts the tables and columns implied by this structure [30]. This recovers latent schema dependencies, such as join tables, grouping columns, or aggregation-related attributes, that may be missed by direct matching.

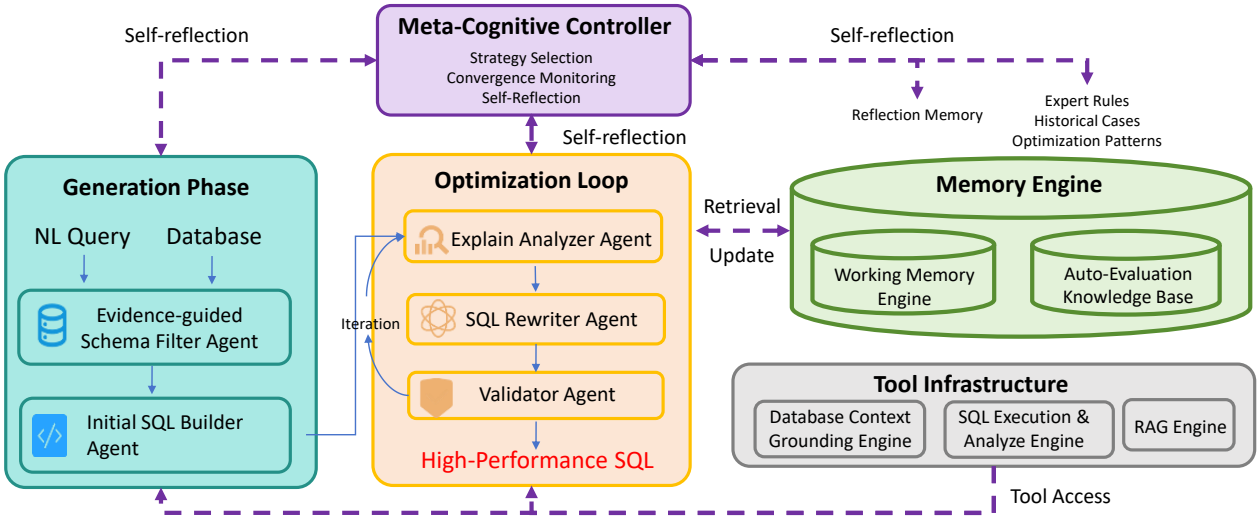

Fig. 2: **Overview of OPTSQL**

- *Value-grounded schema linking* identifies relevant schema elements through actual database contents. Instead of relying only on table or column names, the agent checks whether values in a column match or semantically correspond to phrases in the question, which is especially useful when schema names are ambiguous or weakly informative.

Beyond schema linking, the agent further performs active value grounding to translate natural language expressions into concrete database predicates. It supports several value grounding capabilities:

- *Exact value verification* checks whether a phrase in the question directly appears in candidate columns. This is useful for grounding categorical conditions such as names, locations, statuses, or labels.
- *Semantic value matching* retrieves column values that are semantically close to question phrases, allowing the agent to handle paraphrases, abbreviations, and domain-specific expressions.
- *Distinct-value inspection* examines representative or unique values of a column to infer how natural language concepts are encoded in the database, such as whether a Boolean concept is represented by `0/1`, `Y/N`, or textual labels.
- *Data sampling and distribution checking* inspects example rows or value distributions to understand column semantics and avoid grounding predicates on irrelevant or sparsely populated columns.
- *Null and type analysis* checks whether candidate columns contain nulls or unexpected value formats, helping the agent avoid unreliable predicates and distinguish categorical, numerical, date, and Boolean constraints.
- *Composite predicate grounding* combines multiple grounded values or columns when a natural language phrase corresponds to a compound condition, such as a grade span, date range, or multi-attribute status.

For example, a phrase such as "kindergarten to 8th grade" may be grounded as `Low_Grade = 'K'` and `High_Grade = '8'`, while "magnet program" may correspond to a binary or categorical value after inspecting the value distribution of a related column. These grounded facts are passed to the SQL generator as explicit evidence, reducing the risk of hallucinated predicates or incorrect value choices. Finally, the agent further completes the relational context required for SQL construction. The selected schema elements may be semantically relevant but structurally disconnected; for example, two relevant tables may require intermediate tables or primary–foreign key columns to form a valid join path. Therefore, the agent supplements necessary join keys and bridge tables according to the database relationships, so that the resulting schema context can support executable SQL generation. The output of this agent is an evidence-enriched schema context that integrates schema relevance, verified value grounding, ambiguous-case evidence, and join-path information for downstream SQL generation.

Initial SQL Builder Agent. Initial SQL Builder Agent is responsible for producing executable and semantically plausible initial SQL candidates. It provides a set of complementary SQL construction capabilities, rather than a fixed generation pipeline. Depending on the task complexity, schema ambiguity, and control signals from the Meta-Cognitive Controller, the agent may invoke one or more generation strategies to balance robustness and efficiency. The agent supports the following SQL generation strategies:

- *Skeleton-based generation* follows a top-down construction process. It first analyzes the required SQL components, such as selection, filtering, joining, grouping and ordering, then sketches a SQL skeleton, and finally fills in concrete tables, columns, predicates, and functions. This strategy is useful for queries that require clear structural planning.
- *ICL-based generation* leverages retrieved demonstrations to guide SQL construction. By retrieving structurally similar examples, the agent can reuse successful query patterns and adapt them to the current grounded schema. This strategy is especially helpful when the question resembles previously observed SQL structures.
- *Decomposition-based generation* breaks a complex question into simpler sub-questions and generates SQL fragments for each sub-task before composing them into a complete query.

This strategy is designed for complex analytical questions involving nested reasoning, multiple conditions, or multi-step aggregation.

After candidate generation, the agent can apply a lightweight repair loop to improve executability. It checks basic syntactic and execution errors, and revises invalid candidates using the original question, grounded schema evidence, and database error messages. This repair process removes obvious generation failures before candidate selection, while leaving global strategy switching and deeper optimization to the Meta-Cognitive Controller. When the controller activates multiple generation strategies, the agent may produce multiple candidate SQLs. In this case, the agent further performs *execution-based selection*. Candidates are grouped according to their execution results, and each group is treated as a behavioral cluster. If one cluster forms a dominant majority, the agent selects a representative query from that cluster as the initial SQL. Otherwise, representative candidates from the top clusters are compared through pairwise semantic judgment, and each candidate is ranked by a confidence-aware score. The candidate with the highest score is selected as the output of the Initial SQL Builder Agent.

C. Optimization Loop

The Optimization Loop aims to improve the execution efficiency of an initial SQL query while preserving its semantics. It is activated by the Meta-Cognitive Controller when the generated SQL may benefit from further optimization. The loop consists of three agents: the Explain Analyzer Agent, the SQL Rewriter Agent, and the Validator Agent.

Explain Analyzer Agent. This agent is responsible for diagnosing potential inefficiencies in the current SQL query and producing optimization suggestions for the SQL Rewriter Agent. Given an initial SQL query, it first obtains the query plan from the database and normalizes raw plan operators into stable bottleneck signals. These plan-level signals are then combined with lightweight SQL-structure analysis and schema statistics, such as table size, available indexes, and nullable columns, to decide whether the observed behavior is likely to be a real bottleneck. The agent then maps each bottleneck signal to one or more candidate rewrite directions. For instance, consider a query such as "`SELECT order_id, customer_id, order_date FROM orders WHERE status = 'shipped' ORDER BY order_date LIMIT 10`". If the query plan contains both `SCAN orders` and `USE TEMP B-TREE FOR ORDER BY`, the agent derives two signals: `full_table_scan` and `temp_sort`. When `orders` is a large table, the full scan indicates a potentially expensive access path and may trigger access-path-aware suggestions, while the temporary sort may suggest aligning the `ORDER BY`/`LIMIT` pattern with available ordering or indexes. The output of the agent is therefore a set of suggestions, such as *missing filter pushdown* or *inefficient ordering*, rather than a rewritten SQL query.

As shown in Table I, the agent maintains a set of optimization patterns. Each pattern describes a possible inefficiency, a candidate rewrite direction, and an applicability condition that constrains when the rewrite can be attempted without changing query semantics. These suggestions are not directly applied as deterministic rules. Instead, they serve as evidence for the SQL Rewriter Agent, which decides how to rewrite the query under semantic-preservation constraints.

SQL Rewriter Agent. The SQL Rewriter Agent transforms the current SQL query into a potentially more efficient variant based on the suggestions produced by the Explain Analyzer Agent. Its input includes the original SQL, the detected bottleneck signals, the corresponding rewrite directions, the applicability conditions and optionally retrieved rewrite cases from the Memory Engine. The rewriter does not freely optimize the query; instead, it is constrained to apply rewrite actions that are supported by the available evidence.

Validator Agent. The Validator Agent determines whether a rewritten SQL query should be accepted. It performs two checks. First, it checks semantic consistency between the original SQL and the rewritten SQL, ensuring that the rewrite does not change the intended query result. Second, it evaluates whether the rewritten SQL improves execution efficiency using available database feedback including estimated optimizer cost and execution latency. If the rewritten SQL fails the semantic-consistency check, it is rejected and returned to the Meta-Cognitive Controller for reflection. The controller may then ask the SQL Rewriter Agent to retry with the original SQL, the failed rewrite, and the validation feedback. If the rewritten SQL preserves semantics but does not reduce cost, the controller may retry with another rewrite direction, retrieve additional cases from the Memory Engine, or terminate the optimization loop when further improvement is unlikely. Only rewrites that pass both semantic consistency and cost-improvement checks are accepted as optimized outputs.

D. Meta-Cognitive Controller

The Meta-Cognitive Controller is the central coordination module of OPTSQL. It maintains a global task state, selects which agent capabilities should be invoked, monitors intermediate feedback, and decides whether the system should continue, switch strategy, or terminate. In this sense, the controller provides process-level regulation over the whole Text-to-SQL workflow.

The task state is initialized from the input question and database environment, and is further enriched with relevant knowledge retrieved from the AEKB, which serves as long-term memory for reusable optimization experience. During execution, this task-level state is maintained in working memory and updated after each agent action using structured reports from the Generation Phase and the Optimization Loop. Based on the updated working-memory snapshot, the controller identifies high-level control signals, such as whether the current evidence is sufficient, whether candidate SQLs are stable, whether additional generation strategies are needed, whether the query should be optimized, whether a rewrite is semantically risky, and whether the current optimization loop is converging. These signals provide the basis for work-

TABLE I: **Optimization patterns used by the Explain Analyzer Agent.**

Pattern	Rewrite Direction	Applicability Condition
Redundant projection	Replace `SELECT *` with necessary projected columns.	Applied when required output columns can be inferred from the question or generated SQL.
Scalar `MAX`/`MIN` subquery	Rewrite to `ORDER BY ... LIMIT 1`.	Applied only when tie behavior and aggregation semantics are preserved.
Correlated subquery	Rewrite to `JOIN` or `CTE`.	Applied when an equivalent rewrite pattern is retrieved or can be validated.
Missing filter pushdown	Move selective predicates closer to base tables.	Applied when predicate movement does not change join or aggregation semantics.
Late aggregation	Pre-aggregate before join.	Applied when grouping keys and aggregation semantics remain equivalent.
Inefficient `OR` predicate	Rewrite to `UNION` or `UNION ALL`.	Applied when duplicate semantics are correctly handled.
Redundant join path	Simplify unnecessary joins or bridge tables.	Applied only when projected columns and filtering semantics are unaffected.
Function on indexed column	Rewrite to an equivalent range or comparison predicate.	Applied when the transformation preserves value semantics.
Inefficient ordering	Align `ORDER BY` / `LIMIT` with available ordering or indexes.	Applied when ordering semantics are unchanged.

flow adaptation and process-level self-reflection, guiding the controller to continue, retry, switch strategy, or terminate the current reasoning process.

Workflow and Strategy Adaptation. The controller dynamically selects a workflow according to the current task state. Its adaptation operates at three levels:

- *Evidence-construction strategy selection.* The controller decides how deeply the Evidence-guided Schema Filter Agent should construct schema and value evidence. When schema relevance is clear, basic direct schema linking may be sufficient. When schema relevance is uncertain, the controller can request reverse schema linking to recover latent table or column dependencies. When natural language expressions are ambiguous, it can request value-grounded schema linking or additional value inspection, such as checking exact values, retrieving semantically similar values, sampling column contents, or inspecting null/type information.
- *SQL-construction strategy selection.* The controller decides whether the Initial SQL Builder Agent should use one or multiple generation strategies. Simple queries may use only skeleton-based generation, while complex analytical queries may activate multiple complementary strategies jointly, including ICL-based generation and decomposition-based generation.
- *Optimization strategy selection.* After an initial SQL is obtained, the controller decides whether to enter the Optimization Loop. This decision is based on query structure, database scale, and execution feedback. After optimization is activated, the Explain Analyzer Agent further diagnoses inefficient patterns and produces rewrite suggestions. If optimization is activated, the controller assigns an optimization budget, such as the maximum number of rewrite iterations and retry attempts.

This adaptive design avoids applying the same expensive workflow to every question, while retaining stronger reasoning paths for ambiguous, complex, or performance-sensitive cases.

Process-level Self-Reflection. The controller performs self-reflection when intermediate feedback suggests that the current reasoning path is unreliable, uncertain, or no longer improving. Rather than waiting until the final SQL is produced, it monitors evidence quality, candidate stability, execution behavior, semantic validation results, and optimization progress throughout the workflow. When potential problems are detected, the controller can revise the current plan by requesting additional evidence construction, activating alternative generation strategies, invoking repair, retrieving more relevant optimization cases, or switching to another rewrite direction. During optimization, self-reflection is mainly driven by the Validator Agent's feedback. If a rewritten SQL violates semantic consistency, the controller rejects the rewrite and asks the SQL Rewriter Agent to revise it. If the rewritten SQL preserves semantics but fails to reduce cost, the controller retries with rewrite history, retrieved cases, and optimization suggestions, or switches to another rewrite direction. The controller also terminates the optimization loop when the retry budget is exhausted or the cost reduction remains below a predefined threshold, avoiding unproductive optimization attempts.

E. Memory Engine and Tool Infrastructure

Memory Engine. The Memory Engine supports both task-level reasoning and cross-task experience reuse. It consists of working memory and the Auto-Evaluation Knowledge Base (AEKB). The working memory stores the structured task state and intermediate trace of the current task. It records task context, including the database environment, question complexity, and available statistics or indexes. It also stores grounding evidence, including selected tables and columns, grounded values, ambiguous schema or value alternatives, join-path evidence, and evidence reliability. During SQL generation, it records selected generation strategies, generated SQL candidates, execution errors, repair attempts, execution-result clusters, candidate confidence, and the selected initial SQL. During optimization, it records query plans, normalized bottleneck signals, optimization hints, rewrite directions, applicability conditions, retrieved cases, rewritten SQLs, semantic validation results, estimated costs, execution latency, and cost changes. The Meta-Cognitive Controller uses this memory to avoid repeated failed actions and provide context for subsequent reflection, retry, and termination decisions. Failed rewrites are kept only in working memory as task-level negative feedback, rather than being promoted to reusable knowledge. The AEKB stores reusable optimization knowledge. It contains expert rules and historical rewrite cases validated by the Validator Agent. A rewrite is added to the AEKB only when it satisfies two conditions: the rewritten SQL is semantically consistent with the original SQL, and it reduces execution cost. Each accepted case is summarized as a rewrite template record:

$$\langle T_{src}, T_{dst}, C, R \rangle.$$

Here, (T_{src}) denotes the source SQL template, (T_{dst}) denotes the rewritten SQL template, (C) specifies the applicability constraints, and (R) describes the optimization rationale. To retrieve relevant cases, the RAG Engine first normalizes the current SQL into a template. It parses the SQL into an AST, replaces table names with `TABLE`, replaces column names with coarse semantic tokens such as `TEXT_COLUMN`, `NUMERIC_COLUMN`, `DATE_COLUMN`, `ID_COLUMN`, and

FK_COLUMN, replaces literals with LITERAL, and normalizes whitespace and case. The normalized template is then compared with the (T_{src}) templates in the AEKB using a hybrid similarity score that combines sequence similarity and token-set overlap. The top-ranked records are returned as candidate rewrite knowledge for the SQL Rewriter Agent. Through this mechanism, OPTSQL accumulates validated optimization experience and reuses it to guide future bottleneck analysis and rewrite selection.

Tool Infrastructure. The Tool Infrastructure provides executable interfaces between the agents and the database environment. It consists of three groups of tools:

- *Database Context Grounding Engine.* This engine supports schema and value evidence construction for the Evidence-guided Schema Filter Agent. It provides operations for schema linking, value verification, semantic value matching, distinct-value inspection, data sampling, null/type analysis, and relational completion. These tools help the system ground natural language expressions into relevant tables, columns, values, and join paths.
- *SQL Execution and Analysis Engine.* This engine supports execution-aware optimization for the Optimization Loop. It includes a query plan analyzer, a SQL quality analyzer, a semantic consistency checker, and a cost evaluator. The query plan analyzer normalizes raw query plans into plan-level risk signals, while the SQL quality analyzer detects static SQL risks from the parsed AST. The semantic consistency checker compares the original and rewritten SQLs on the available database state, and the cost evaluator measures estimated optimizer cost and execution latency before and after rewriting.
- *RAG Engine.* This engine supports memory retrieval and update. It provides a SQL formatter for template normalization, a retriever for selecting relevant rewrite cases, and an updater for inserting validated optimization cases into the AEKB. These tools allow the SQL Rewriter Agent to reuse prior optimization experience and update the knowledge base after successful rewrites.

Together, these tools allow OPTSQL to ground reasoning in concrete database evidence, analyze execution behavior, retrieve relevant optimization experience, and update memory after successful rewrites.

IV. CASE STUDY

V. EXPERIMENT

VI. DISCUSSION

VII. RELATED WORK

Fine-tuned and Trained Text-to-SQL Models. A dominant line of recent work focuses on adapting or training LLMs for Text-to-SQL via supervised fine-tuning, synthetic data construction, or reinforcement learning [31], [19], [32], [33], [34], [35], [36], [37]. For example, XIYAN-SQL [36] uses multi-task fine-tuning and a multi-generator ensemble to produce diverse SQL candidates, while OMNISQL [37] improves model training through large-scale synthesized Text-to-SQL data. Reinforcement-learning-based methods such as REASONING-SQL [38] and ARCTIC-TEXT2SQL-R1 [39] further train models with SQL-tailored rewards or execution-based feedback to improve reasoning and executability. Despite their strong performance on benchmarks, these methods remain heavily dependent on training data quality and coverage. They often exhibit limited generalization in out-of-domain scenarios and may suffer from reduced robustness when facing complex or real-world database environments [30].

LLM Prompting and Retrieval-augmented Generation. Another line of work leverages general-purpose LLMs for Text-to-SQL through prompting and retrieval-augmented generation, performing task adaptation at inference time without parameter updates. These methods formulate SQL generation as an in-context reasoning problem, where the model is guided by schema serialization, database content, demonstrations and decomposition instructions. Early benchmark and adaptation studies show that LLMs can become strong Text-to-SQL generators when prompts provide sufficient database context and carefully selected examples [4], [16], [40]. Subsequent prompting methods improve robustness through task decomposition, self-correction, dynamic few-shot selection, and consistency alignment [5], [41], [42]. Retrieval-augmented approaches further strengthen grounding by retrieving relevant demonstrations, linking schema elements at scale, pruning database context, and selecting among multiple candidates [8], [7], [43], [44], [45]. Despite these advances, prompting and retrieval-based methods still depend heavily on manually designed inference procedures and the quality of retrieved evidence. Their reasoning depth is usually determined before generation, which limits their ability to adapt the workflow according to schema scale, intermediate uncertainty, or execution behavior [30], [7].

Agentic Text-to-SQL Workflows. Recent work further extends LLM prompting into agentic workflows, where LLMs call tools, inspect database context, execute SQL, observe feedback, and revise intermediate outputs. General agent frameworks such as REACT [15], SELF-REFINE [46], and REFLEXION [47] demonstrate the value of combining reasoning, acting, and iterative self-improvement. Following this paradigm, Text-to-SQL agents organize generation around modules such as schema exploration, candidate generation, execution-based repair, multi-path reasoning, semantic memory, and agentic view construction [6], [43], [48], [11], [12], [13], [9], [10]. These systems are more robust than single-shot prompting because they can query the database, compare alternatives, and use observed failures to revise SQL. Nevertheless, most existing agents still instantiate a largely fixed workflow: similar modules are invoked for most questions, and database feedback is mainly used for post-generation repair, validation, or candidate selection rather than for continuous process control.

Difference from OPTSQL. Existing prompting-based and agentic Text-to-SQL methods mainly focus on improving correctness through better grounding, decomposition, and post-

hoc refinement. In contrast, OPTSQL is designed as an efficiency- and process-aware framework that goes beyond static or fixed workflows. It introduces a meta-cognitive controller that enables (i) environment-aware workflow adaptation, (ii) process-level self-reflection during generation, and (iii) execution- and plan-aware optimization guided by database feedback. As a result, OPTSQL treats Text-to-SQL as a dynamic interaction process where both reasoning strategy and optimization behavior are continuously adjusted according to the database environment and intermediate execution signals, rather than following a predefined pipeline or relying only on post-generation correction.

VIII. CONCLUSION

REFERENCES

[1] I. Androutsopoulos, G. D. Ritchie, and P. Thanisch, "Natural language interfaces to databases–an introduction," *Natural language engineering*, vol. 1, no. 1, pp. 29–81, 1995.

[2] G. G. Hendrix, E. D. Sacerdoti, D. Sagalowicz, and J. Slocum, "Developing a natural language interface to complex data," *ACM Transactions on Database Systems (TODS)*, vol. 3, no. 2, pp. 105–147, 1978.

[3] A.-M. Popescu, O. Etzioni, and H. Kautz, "Towards a theory of natural language interfaces to databases," in *Proceedings of the 8th international conference on Intelligent user interfaces*, 2003, pp. 149–157.

[4] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," *arXiv preprint arXiv:2308.15363*, 2023.

[5] M. Pourreza and D. Rafiei, "Din-sql: Decomposed in-context learning of text-to-sql with self-correction," *Advances in Neural Information Processing Systems*, vol. 36, pp. 36 339–36 348, 2023.

[6] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun *et al.*, "Mac-sql: A multi-agent collaborative framework for text-to-sql," *arXiv preprint arXiv:2312.11242*, 2023.

[7] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "Chess: Contextual harnessing for efficient sql synthesis," *arXiv preprint arXiv:2405.16755*, 2024.

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[9] M. T. Pham, T. Pham, T. Chen, H. Yin, Q. V. H. Nguyen, and T. T. Nguyen, "Av-sql: Decomposing complex text-to-sql queries with agentic views," *arXiv preprint arXiv:2604.07041*, 2026.

[10] A. Biswal, C. Lei, X. Qin, A. Li, B. Narayanaswamy, and T. Kraska, "Agentsm: Semantic memory for agentic text-to-sql," *arXiv preprint arXiv:2601.15709*, 2026.

[11] S. Chaturvedi, A. Chadha, and L. Bindschaedler, "Sql-of-thought: Multi-agentic text-to-sql with guided error correction," *arXiv preprint arXiv:2509.00581*, 2025.

[12] O. R. Heidari, S. Reid, and Y. Yaakoubi, "Agentiql: An agent-inspired multi-expert framework for text-to-sql generation," *arXiv preprint arXiv:2510.10661*, 2025.

[13] H. Yang, J. Zhang, Z. He, and Y. R. Fung, "Mars-sql: A multi-agent reinforcement learning framework for text-to-sql," *arXiv preprint arXiv:2511.01008*, 2025.

[14] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[16] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo *et al.*, "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," *Advances in Neural Information Processing Systems*, vol. 36, pp. 42 330–42 357, 2023.

[17] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows," *arXiv preprint arXiv:2411.07763*, 2024.

[18] H. Li, J. Zhang, C. Li, and H. Chen, "Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13 067–13 075.

[19] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "Codes: Towards building open-source language models for text-to-sql," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–28, 2024.

[20] J. Li, B. Hui, R. Cheng, B. Qin, C. Ma, N. Huo, F. Huang, W. Du, L. Si, and Y. Li, "Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 11, 2023, pp. 13 076–13 084.

[21] D. Rai, B. Wang, Y. Zhou, and Z. Yao, "Improving generalization in language model-based text-to-sql semantic parsing: Two simple semantic boundary-based techniques," *arXiv preprint arXiv:2305.17378*, 2023.

[22] T. Scholak, N. Schucher, and D. Bahdanau, "Picard: Parsing incrementally for constrained auto-regressive decoding from language models," *arXiv preprint arXiv:2109.05093*, 2021.

[23] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers," *arXiv preprint arXiv:1911.04942*, 2019.

[24] S. Chaudhuri, "An overview of query optimization in relational systems," in *Proceedings of the seventeenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*, 1998, pp. 34–43.

[25] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015.

[26] Z. Wang, Z. Zhou, Y. Yang, H. Ding, G. Hu, D. Ding, C. Tang, H. Chen, and J. Li, "Wetune: Automatic discovery and verification of query rewrite rules," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 94–107.

[27] L. Lin, Y. Shen, L. Bao, R. Wu, and Y. Liu, "Cracking query bottlenecks: Towards efficiency-oriented text-to-sql generation," in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2026.

[28] F. J. Varela, E. Thompson, and E. Rosch, *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press, 1991.

[29] J. H. Flavell, "Metacognition and cognitive monitoring: A new area of cognitive-developmental inquiry," *American Psychologist*, vol. 34, no. 10, pp. 906–911, 1979.

[30] B. Li, C. Chen, Z. Xue, Y. Mei, and Y. Luo, "Deepeye-sql: A software-engineering-inspired text-to-sql framework," *Proceedings of the ACM on Management of Data*, vol. 4, no. 3 (SIGMOD, pp. 1–28, 2026.

[31] R. Sun, S. Ö. Arik, A. Muzio, L. Miculicich, S. Gundabathula, P. Yin, H. Dai, H. Nakhost, R. Sinha, Z. Wang, and T. Pfister, "SQL-PaLM: Improved large language model adaptation for text-to-sql," *arXiv preprint arXiv:2306.00739*, 2023.

[32] M. Pourreza and D. Rafiei, "DTS-SQL: Decomposed text-to-sql with small large language models," in *Findings of the Association for Computational Linguistics: EMNLP*, 2024.

[33] X. Chen, T. Wang, T. Qiu, J. Qin, and M. Yang, "Open-SQL framework: Enhancing text-to-sql on open-source large language models," *arXiv preprint arXiv:2405.06674*, 2024.

[34] D. G. Thorpe, A. J. Duberstein, and I. A. Kinsey, "Dubo-SQL: Diverse retrieval-augmented generation and fine tuning for text-to-sql," *arXiv preprint arXiv:2404.12560*, 2024.

[35] P. Ma, X. Zhuang, C. Xu, X. Jiang, R. Chen, and J. Guo, "SQL-R1: Training natural language to sql reasoning model by reinforcement learning," *arXiv preprint arXiv:2504.08600*, 2025.

[36] Y. Liu, Y. Zhu, Y. Gao, Z. Luo, X. Li, X. Shi, Y. Hong, J. Gao, Y. Li, B. Ding, and J. Zhou, "XiYan-SQL: A novel multi-generator framework for text-to-sql," *arXiv preprint arXiv:2507.04701*, 2025.

[37] H. Li, S. Wu, X. Zhang, X. Huang, J. Zhang, F. Jiang, S. Wang, T. Zhang, J. Chen, R. Shi *et al.*, "Omnisql: Synthesizing high-quality text-to-sql data at scale," *arXiv preprint arXiv:2503.02240*, 2025.

[38] M. Pourreza, S. Talaei, R. Sun, X. Wan, H. Li, A. Mirhoseini, A. Saberi, S. O. Arik *et al.*, "Reasoning-sql: Reinforcement learning with sql tailored partial rewards for reasoning-enhanced text-to-sql," *arXiv preprint arXiv:2503.23157*, 2025.

[39] Z. Yao, G. Sun, L. Borchmann, Z. Shen, M. Deng, B. Zhai, H. Zhang, A. Li, and Y. He, "Arctic-Text2SQL-R1: Simple rewards, strong reasoning in text-to-sql," *arXiv preprint arXiv:2505.20315*, 2025.

[40] R. Sun, S. Ö. Arik, A. Muzio, L. Miculicich, S. Gundabathula, P. Yin, H. Dai, H. Nakhost, R. Sinha, Z. Wang *et al.*, "Sql-palm: Improved large language model adaptation for text-to-sql (extended)," *arXiv preprint arXiv:2306.00739*, 2023.

[41] X. Xie, G. Xu, L. Zhao, and R. Guo, "Opensearch-sql: Enhancing text-to-sql with dynamic few-shot and consistency alignment," *arXiv preprint arXiv:2502.14913*, 2025.

[42] A. Ramachandran and S. Sarawagi, "Text-to-sql calibration: No need to ask – just rescale model probabilities," *arXiv preprint arXiv:2411.16742*, 2024.

[43] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Özcan, and S. O. Arik, "Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql," *arXiv preprint arXiv:2410.01943*, 2024.

[44] Y. Wang, P. Liu, and X. Yang, "Linkalign: Scalable schema linking for real-world large-scale multi-database text-to-sql," *arXiv preprint arXiv:2503.18596*, 2025.

[45] K. Maamari, F. Abubaker, D. Jaroslawicz, and A. Mhedhbi, "The death of schema linking? text-to-sql in the age of well-reasoned language models," *arXiv preprint arXiv:2408.07702*, 2024.

[46] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 534–46 594, 2023.

[47] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652.

[48] M. Deng, A. Ramachandran, C. Xu, L. Hu, Z. Yao, A. Datta, and H. Zhang, "Reforce: A text-to-sql agent with self-refinement, consensus enforcement, and column exploration," *arXiv preprint arXiv:2502.00675*, 2025.